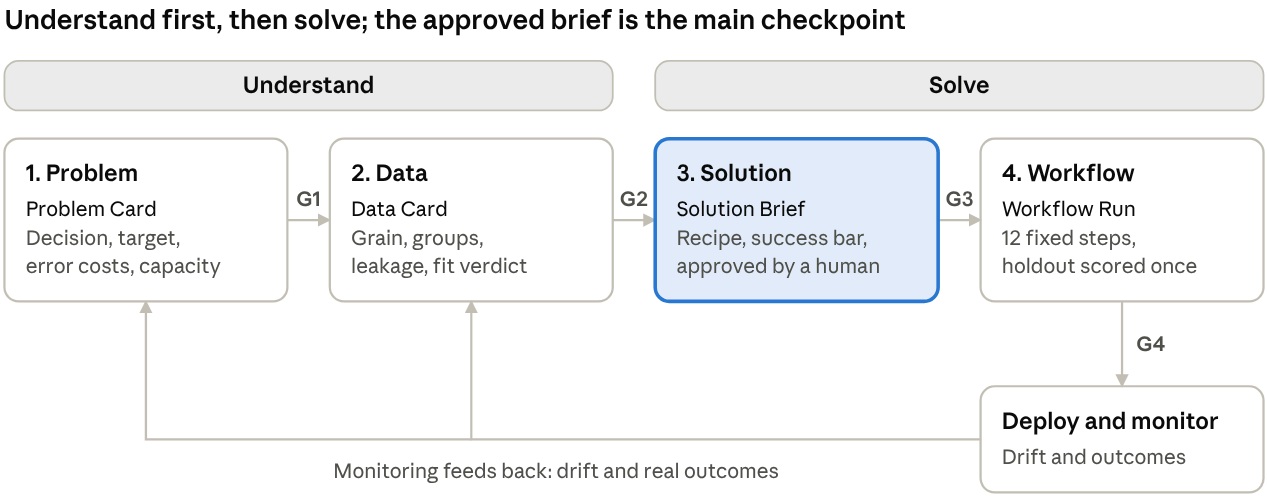
DCLab Framework: From Problem to Solution Workflow

Oct 16, 2026 · @Shahriar

Overview

Every ML problem in DCLab goes through two stages and four phases. **Understand** covers the problem and the data. **Solve** covers the solution and the workflow that carries it out. Each phase produces one structured card, and each ends with a gate that must pass before the next phase starts.



two stages, four phases, four gates, one feedback loop

Work only moves right when a gate passes; a failed gate sends it back to an earlier phase with a reason (see Gates and loops). The approved Solution Brief is the main human checkpoint. After it, the workflow runs all its steps without stopping.

Stage	Phase	Question it answers	Output
Understand	1. Problem	What decision are we improving, and what exactly do we predict?	Problem Card
Understand	2. Data	Can this data answer that question honestly?	Data Card + fit verdict

Stage	Phase	Question it answers	Output
Solve	3. Solution	What approach, judged by what success bar?	Solution Brief
Solve	4. Workflow	Carry it out step by step, with checks	Workflow Run (artifact + report)

Phase 1: Problem understanding

This phase turns a business request into a precise ML question. Most failed ML projects fail here: the model answers a question nobody asked, or predicts something no one can act on. The output is a **Problem Card**.

Questions to answer, in order:

1. **Decision:** which business decision will this improve, and who makes it?
2. **Action:** what will that person do differently based on the output?
3. **Unit:** what does one prediction describe? One customer, transaction, store-day, pair of records?
4. **Target:** exactly what event or number is predicted, with the definition written so two people would label it the same way.
5. **Timing:** when is the prediction made, and how far ahead does it look (the horizon)?
6. **Cost of errors:** what does a false alarm cost, and what does a miss cost? Which is worse?
7. **Constraints:** capacity (for example, cases reviewed per day), budget, speed, regulation, fairness.
8. **Success:** the business metric that must move, and the ML metric that stands in for it.
9. **Baseline:** what happens today without the model, and how well does it work?

The Problem Card:

Field	What it holds
`decision`	The business decision and its owner
`action`	What changes when the output arrives
`unit`	The entity one prediction describes

Field	What it holds
`target_definition`	The exact event or quantity, in one sentence
`prediction_time`, `horizon`	When the prediction is made and how far ahead it looks
`error_costs`	Cost of a false positive versus a false negative
`constraints`	Capacity K, budget, latency, regulation
`success_metric`	Business metric, and the ML metric that stands in for it
`baseline_today`	The current process and its known performance
`framing_hint`	First guess of task kind and modifiers (confirmed in Phase 2)
`open_questions`	Anything still unknown, with who can answer it

Done when every field is filled or listed as an open question with an owner, and the user confirms the target definition.

Phase 2: Data understanding

This phase checks whether the data can actually answer the question from Phase 1. It is not general EDA. Every check is read against the Problem Card. The output is a **Data Card**, plus a verdict on problem-data fit.

Checks, in order:

1. **Grain:** what one row is (which entity, at which time) and whether it matches the Problem Card's `unit`.
2. **Target:** that the target column exists, matches `target_definition`, and how rare each class is.
3. **Time:** the date range, the time column, and whether it covers the `horizon` with enough history.
4. **Keys and groups:** repeated IDs, near-duplicate rows, and entities that appear many times.
5. **Columns:** type, missing rate, number of unique values, and semantic role (identifier, feature, target, timestamp, label source).

6. **Availability:** for each column, whether its value is known at `prediction_time`. A column recorded after the outcome is leakage.
7. **Labels:** where labels come from (system, rule, expert) and how much they can be trusted.
8. **Stability:** whether distributions or the target rate change over time (drift).
9. **Volume:** whether there are enough rows and enough positives for the task kind.

The Data Card:

Field	What it holds
`sources`	Tables or files, with row counts and date ranges
`grain`	What one row represents
`target_profile`	Target column, class rates or distribution, label sources
`time_profile`	Time column, date range, target rate over time
`groups`	Repeated keys and their counts
`columns`	Per column: type, missing %, unique count, semantic role, available at prediction time (yes/no)
`leakage_suspects`	Columns flagged, each with the reason
`quality_issues`	Duplicates, impossible values, broken joins
`detected_modifiers`	Modifiers the data shows, each with evidence
`fit_verdict`	Fits, fits with changes, or does not fit, and why

Problem-data fit. Phase 2 ends with one of three verdicts:

- **Fits:** go to Phase 3.
- **Fits with changes:** the problem must be adjusted to the data (for example, a shorter horizon because history is short, or a different target because the planned one is not recorded). Update the Problem Card, then go to Phase 3.
- **Does not fit:** stop and say what data is missing. A model trained on data that cannot answer the question is worse than no model.

Phase 3: Solution understanding

This phase decides **how** to solve the problem before any training starts. It reads both cards and produces a **Solution Brief**: the chosen approach, why it was chosen, and what will count as success. Writing success criteria down before training stops the team from moving the goalposts after seeing results.

Questions to answer:

1. **Is ML needed at all?** Check the simplest option first: a rule or script, then classical ML, then a small language model, then a frontier LLM. Stop at the first that can meet the success metric.
2. **What is the recipe?** One task kind plus its modifiers, taken from the Data Card's evidence (see DCLab Task Recipes).
3. **What is the baseline to beat?** Today's process from the Problem Card, plus the recipe's mandatory simple model.
4. **How will it be validated?** Split type, holdout period, and the rule that the holdout is scored only once.
5. **What is the success bar?** The primary metric value that must be reached, and the guard metrics that must not get worse.
6. **What is the decision rule?** How a score becomes an action: a threshold, a top-K list, or a ranking.
7. **What could go wrong?** Main risks and assumptions, each with how it will be checked.
8. **What else was considered?** At least one alternative approach and why it was not chosen.

The Solution Brief:

Field	What it holds
`approach_level`	Rule, classical ML, small model, or LLM, and why simpler levels are not enough
`recipe`	Task kind, modifiers, and recipe versions
`baselines`	Current process and simple model, with their expected scores
`validation_plan`	Split, folds, holdout period, "score holdout once" rule

Field	What it holds
`success_criteria`	Primary metric target, guard metrics, business metric
`decision_rule`	Threshold, top-K or ranking, and how it was set
`feature_plan`	Feature groups to build, and columns excluded with reasons
`risks`	Each risk with the check that will catch it
`alternatives`	Options considered and why they lost
`approval`	Who approved the brief, and when

Done when the brief is approved. In DCLab this is the main human checkpoint: after approval, Phase 4 can run end to end without stopping.

Phase 4: Solution workflow

This phase carries out the approved Solution Brief as a fixed sequence of steps. Each step has a clear input, output and check, so the workflow can run without stopping and every result can be traced back to a step. The output is a **Workflow Run**: the trained artifact, the report and the full step log.

#	Step	Input	Output	Check before moving on
1	Lock the split	Data, `validation_plan`	Train, validation and sealed holdout sets	No group or time overlap across sets
2	Clean	Train set, Data Card	Cleaning rules fitted on train only	Rules learned only from train
3	Baseline	Clean train set	Baseline scores	Baseline runs and scores are recorded
4	Build features	`feature_plan`	Feature sets with lineage	Every feature available at prediction time

#	Step	Input	Output	Check before moving on
5	Train candidates	Feature sets, recipe families	Candidate models with out-of-fold predictions	No candidate trained on validation or holdout
6	Evaluate	Out-of-fold predictions	Scores per candidate on primary and guard metrics	Same metric code for all candidates and the baseline
7	Select or ensemble	Scores, diversity	Final model or ensemble	Beats the best single model, or the single model is kept
8	Calibrate and set the rule	Final model, `decision_rule`	Calibration map, threshold or K	Rule fitted on validation, not holdout
9	Score holdout once	Sealed holdout	Final honest scores	Run once; the result is reported even if it is bad
10	Package the artifact	Model, preprocessing, rule	Self-sufficient artifact	Save, reload, predict and match passes
11	Report	All step logs	Report against `success_criteria`	Every claim links to a step result
12	Plan monitoring	Data Card, artifact	Drift baselines and alert thresholds	Thresholds are written into the artifact

If the holdout result misses the success bar, the workflow does not tune again on the holdout. It goes back to Phase 3 with what was learned, and a new brief gets a new sealed holdout or a later time period.

Gates and loops

There are four gates, one at the end of each phase. A failed gate always sends the work back to a named earlier phase with a stated reason; it never just retries the same step.

Gate	Passes when	If it fails, go back to
G1: Problem is clear	Target definition confirmed by the user; every Problem Card field filled or owned as an open question	Phase 1, with the questions still open
G2: Data fits the problem	Grain matches unit, target matches definition, enough positives and history, no blocking leakage	Phase 1 if the problem must change; stop if the data is missing
G3: Solution approved	Brief approved, success bar written before training, at least one alternative recorded	Phase 3; or Phase 2 if a new data check is needed
G4: Result is honest and usable	Holdout scored once, artifact passes reload test, report links every claim to a step	Phase 3 with what was learned; never retune on the holdout

After deployment, monitoring results feed back into Phase 2 (drift changes the Data Card) and into Phase 1 (real outcomes may show the target or the action was wrong). That keeps the framework a loop rather than a one-way process.

Who does what

The split follows one rule: **the LLM is strongest at understanding, rules are strongest at execution, and people own the decisions that carry business risk.** That is why the LLM does most of its work in Phases 1 and 3, and almost none in Phase 4.

Phase	Rules (deterministic code)	LLM	Human
1. Problem	Check that every card field is filled	Turns the user's words into a draft Problem Card; asks the missing questions	Confirms the target definition, error costs and capacity
2. Data	All profiling, leakage timing checks, group and drift detection	Proposes semantic roles from column names; explains findings in plain words	Confirms ambiguous roles and group keys; accepts a "fits with changes" verdict
3. Solution	Resolves the recipe; fills baselines and validation from it	Drafts the brief, alternatives and risks; critiques its own draft	Approves the brief (the main checkpoint)
4. Workflow	Runs every step and every check	Writes the report text from step results; never changes numbers	Reads the report; decides whether to deploy

Two consequences for DCLab:

- With the LLM turned off, Phases 2 and 4 still give the same result. Phases 1 and 3 fall back to a form the user fills in.
- The LLM's outputs in Phases 1 and 3 are exactly the reasoning DCLab wants its own workflow model to learn (see the last section).

Worked example: retail returns fraud

The request, as a retail Loss Prevention team might phrase it: "Staff file fake returns on copied bills. Help us catch them." Below is what each phase produces. The numbers are illustrative.

Phase 1, Problem Card (key fields):

- `decision`: which returns the Loss Prevention team reviews each day.
- `unit`: one return transaction.
- `target_definition`: a return later confirmed as fraudulent by the Loss Prevention team.

- `prediction_time`, `horizon`: when the return is filed; the label arrives within 30 days.
- `error_costs`: a miss loses the refund amount; a false alarm costs about 10 minutes of reviewer time.
- `constraints`: K = 300 reviews per day.
- `success_metric`: confirmed fraud per review hour; precision@300 stands in for it.
- `baseline_today`: 10 to 20 business rules, such as "return filed after 11 p.m."

Phase 2, Data Card (key findings):

- Grain matches: one row per return, linked to its original sale by `bill_id`.
- Target is rare: 1.2% confirmed fraud; labels come from rules, experts and clustering.
- `bill_id` and `cashier_id` repeat: the groups modifier is on.
- Fraud rate rises over the last 3 months: the time-order and drift modifiers are on.
- Leakage suspect: `review_status` is written after the review, so it is excluded.
- Verdict: **fits with changes**. Evaluate only on expert-confirmed labels, because rule-based labels would reward copying the rules.

Phase 3, Solution Brief (key decisions):

- Approach level: classical ML. Rules alone are today's baseline, and an LLM is not needed for scoring.
- Recipe: binary + rare positives + groups + time order + noisy labels + capacity K.
- Validation: last 3 months as the sealed holdout, grouped by `bill_id`.
- Success bar: precision@300 at least 1.5 times the rules baseline, with no drop in recall on the audit sample.
- Alternative considered: unsupervised anomaly scoring only. Rejected because 70,000 labels already exist.

Phase 4, Workflow Run: the 12 steps run without stopping. The holdout is scored once, the artifact carries the threshold for K = 300 and the `bill_id` group key, and the report compares precision@300 with the rules baseline.

How it fits DCLab

The framework gives a shape to pieces DCLab already has, and fills two gaps: there is no Problem Card today, and no Solution Brief that is approved before training.

Framework piece	What DCLab already has	What is new
Phase 1: Problem Card	`target.column` and `task.kind` decision points	The card itself, and the questions about decision, action, error costs and capacity
Phase 2: Data Card	Profiling, column-type, missing-value and leakage decision points	One stored card per run, plus the fit verdict
Phase 3: Solution Brief	Recipe resolution (see DCLab Task Recipes)	Success bar written before training, alternatives, an approval step
Phase 4: Workflow Run	The deterministic pipeline, decision records and artifact export	A fixed 12-step contract with a check per step

For R&D and the workflow model. The four cards are the natural shape for training samples. One sample = request → Problem Card → Data Card → Solution Brief → workflow plan. That is the "problem → workflows → solution → pipeline design" reasoning planned for the workflow LLM, and the approved brief is the point after which it should run all steps without stopping.

- **SFT samples:** each card becomes a target the model learns to write; gate failures become critique samples ("this brief has no success bar; what is missing?").
- **Decision samples:** card fields map to typed decision points, so they produce labels for DCLab's own decision models.
- **Twin runs:** compare the workflow and the Claude agent card by card, not only on the final score. This shows *where* the two lanes first disagree.

Build steps:

1. Define the four card schemas (Pydantic), stored with each run.
2. Fill the Data Card from existing profiling; add the fit verdict.
3. Add the Problem Card as a short guided form, with the LLM drafting it from the user's request.
4. Add the Solution Brief with recipe resolution and an approval step.
5. Map the 12 workflow steps onto the existing pipeline and add the missing checks.
6. Export cards from every run as SFT and decision samples.

خلاصهٔ فارسی

این چارچوب کار روی هر مسئلهٔ ML را در دو بخش و چهار فاز سازمان می‌دهد: بخش «فهم» شامل فهم مسئله و فهم داده است، و بخش «حل» شامل فهم راه‌حل و اجرای گردش‌کار راه‌حل. خروجی هر فاز یک کارت ساختاریافته است و پایان هر فاز یک دروازهٔ بررسی دارد.

فاز	هدف	خروجی
۱. فهم مسئله	تبدیل درخواست کسب‌وکار به یک پرسش ML دقیق	کارت مسئله (Problem Card)
۲. فهم داده	بررسی اینکه داده واقعاً می‌تواند به همان پرسش پاسخ دهد	کارت داده (Data Card) و حکم تناسب مسئله و داده
۳. فهم راه‌حل	انتخاب روش و تعیین معیار موفقیت پیش از آموزش	خلاصهٔ راه‌حل (Solution Brief) با تأیید انسانی
۴. گردش‌کار راه‌حل	اجرای ۱۲ گام ثابت بدون توقف	فایل مدل، گزارش و سابقهٔ کامل گام‌ها

نکته‌های کلیدی:

- بیشتر پروژه‌های ناموفق در فاز اول شکست می‌خورند؛ بنابراین تعریف هدف، هزینهٔ خطاها و ظرفیت بررسی باید پیش از هر کاری مشخص شوند.
- فهم داده یک EDA عمومی نیست؛ هر بررسی در برابر کارت مسئله خوانده می‌شود.
- معیار موفقیت پیش از آموزش نوشته می‌شود و داده‌های ارزیابی نهایی (Holdout) فقط یک بار امتیازدهی می‌شوند.
- تأیید خلاصهٔ راه‌حل نقطهٔ بررسی اصلی انسانی است؛ پس از آن فاز ۴ بدون توقف اجرا می‌شود.
- هر دروازهٔ ناموفق، کار را با دلیل مشخص به یک فاز قبلی برمی‌گرداند.
- این چهار کارت DCLab هستند: درخواست ← کارت مسئله ← کارت داده ← خلاصهٔ راه‌حل ← برنامهٔ گردش‌کار.